Data Loading, Storage and File Formats

Lab 1 - Sales Data

Aim

To load sales data from CSV, Excel and JSON files, clean the data, analyze it and create simple visualizations.

Dataset: `sklearn.datasets.load_wine()`

The sklearn Wine dataset is used to create a small sales-style dataset for this lab.

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine

# Ensure display works in all notebook environments
try:
    from IPython.display import display
except ImportError:
    pass

print("Libraries imported successfully.")
```

Output [1]:

```
Libraries imported successfully.
```

1. Create the sales dataset using sklearn

In [2]:

```
wine = load_wine(as_frame=True)
wine_df = wine.frame

np.random.seed(42)

df = pd.DataFrame({
    "Product": wine_df["target"].map({
        0: "Product A",
        1: "Product B",
        2: "Product C"
    }),
    "Region": np.random.choice(["North", "South", "East", "West"], len(wine_df)),
    "Quantity": np.random.randint(1, 10, len(wine_df)),
    "Unit_Price": np.random.randint(100, 1000, len(wine_df))
})

df["Total_Sales"] = df["Quantity"] * df["Unit_Price"]

# Save the same data in three formats
df.to_csv("sales.csv", index=False)
df.to_excel("sales.xlsx", index=False)
df.to_json("sales.json", orient="records")

print("Files created successfully.")
display(df.head())
```

Output [2]:

```
Files created successfully.
```

Table [2]:

	Product	Region	Quantity	Unit_Price	Total_Sales
0	Product A	East	1	991	991
1	Product A	West	3	844	2532
2	Product A	North	5	136	680
3	Product A	East	3	379	1137
4	Product A	East	1	448	448

2. Load CSV, Excel and JSON files

In [3]:

```
csv_data = pd.read_csv("sales.csv")
excel_data = pd.read_excel("sales.xlsx")
json_data = pd.read_json("sales.json")

print("CSV:")
display(csv_data.head())

print("Excel:")
display(excel_data.head())

print("JSON:")
display(json_data.head())
```

Output [3]:

```
CSV:
Excel:
JSON:
```

Table [3]:

	Product	Region	Quantity	Unit_Price	Total_Sales
0	Product A	East	1	991	991
1	Product A	West	3	844	2532
2	Product A	North	5	136	680
3	Product A	East	3	379	1137
4	Product A	East	1	448	448

Table [3]:

	Product	Region	Quantity	Unit_Price	Total_Sales
0	Product A	East	1	991	991
1	Product A	West	3	844	2532
2	Product A	North	5	136	680
3	Product A	East	3	379	1137
4	Product A	East	1	448	448

Table [3]:

	Product	Region	Quantity	Unit_Price	Total_Sales
0	Product A	East	1	991	991
1	Product A	West	3	844	2532
2	Product A	North	5	136	680
3	Product A	East	3	379	1137
4	Product A	East	1	448	448

3. Explore the data

In [4]:

```
print("Shape:", csv_data.shape)

print("\nColumn names:")
print(csv_data.columns.tolist())

print("\nData types:")
print(csv_data.dtypes)

print("\nMissing values:")
print(csv_data.isnull().sum())

print("\nDuplicate rows:", csv_data.duplicated().sum())
```

Output [4]:

```
Shape: (178, 5)

Column names:
['Product', 'Region', 'Quantity', 'Unit_Price', 'Total_Sales']
```

```
Data types:
Product          str
Region           str
Quantity       int64
Unit_Price     int64
Total_Sales    int64
dtype: object

Missing values:
Product        0
Region         0
Quantity       0
Unit_Price     0
Total_Sales    0
dtype: int64

Duplicate rows: 0
```

4. Clean the data

In [5]:

```
# Remove duplicate rows
df = csv_data.drop_duplicates()

# Fill missing values
df["Region"] = df["Region"].fillna("Unknown")
df["Unit_Price"] = df["Unit_Price"].fillna(df["Unit_Price"].mean())

# Recalculate total sales
df["Total_Sales"] = df["Quantity"] * df["Unit_Price"]

print("Data cleaned successfully.")
display(df.head())
```

Output [5]:

```
Data cleaned successfully.
```

Table [5]:

	Product	Region	Quantity	Unit_Price	Total_Sales
0	Product A	East	1	991	991
1	Product A	West	3	844	2532
2	Product A	North	5	136	680
3	Product A	East	3	379	1137
4	Product A	East	1	448	448

5. Analyze the sales data

In [6]:

```
print("Total Sales:", df["Total_Sales"].sum())
print("Average Sales:", df["Total_Sales"].mean())

print("\nSales by Region:")
sales_region = df.groupby("Region")["Total_Sales"].sum()
display(sales_region)

print("\nSales by Product:")
sales_product = df.groupby("Product")["Total_Sales"].sum()
display(sales_product)
```

Output [6]:

```
Total Sales: 480637
Average Sales: 2700.2078651685392

Sales by Region:

Sales by Product:
```

Output [6]:

```
Region
East      93310
North    121057
South    130506
```

```
West     135764
Name: Total_Sales, dtype: int64
```

Output [6]:

```
Product
Product A    166509
Product B    200847
Product C    113281
Name: Total_Sales, dtype: int64
```

6. Bar chart - Sales by Region

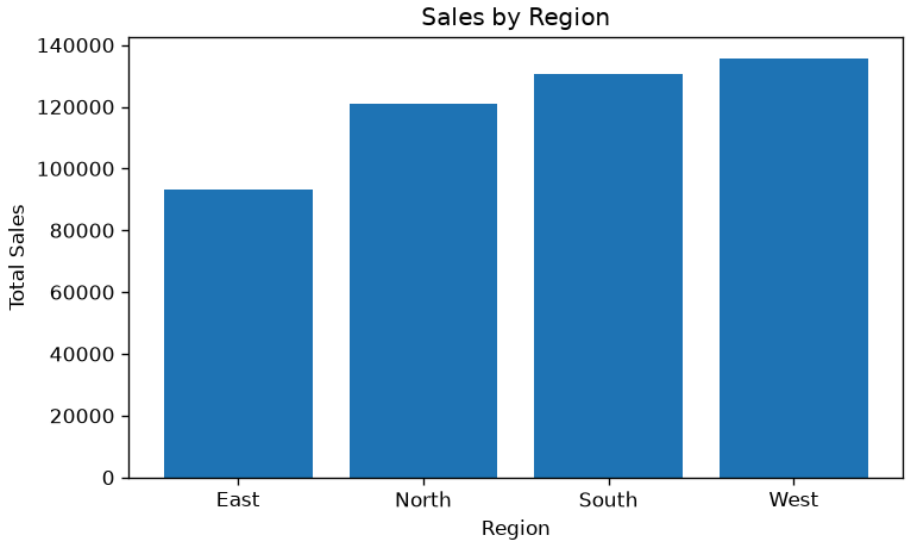
In [7]:

```
plt.figure(figsize=(7, 4))

plt.bar(sales_region.index, sales_region.values)

plt.title("Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.show()
```

Plot [7]:



7. Pie chart - Sales by Product

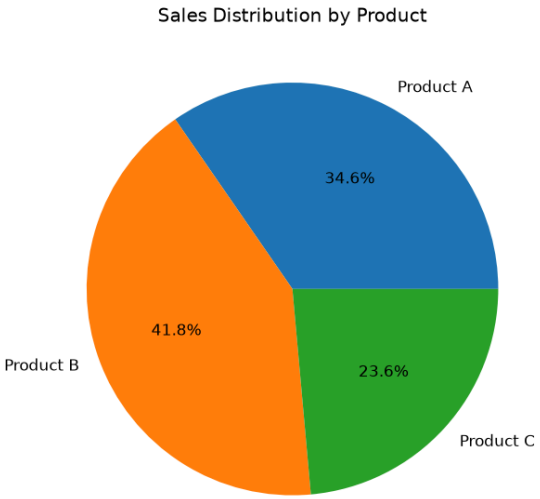
In [8]:

```
plt.figure(figsize=(6, 6))

plt.pie(
    sales_product.values,
    labels=sales_product.index,
    autopct="%1.1f%%"
)

plt.title("Sales Distribution by Product")
plt.show()
```

Plot [8]:



8. Box plot - Total Sales

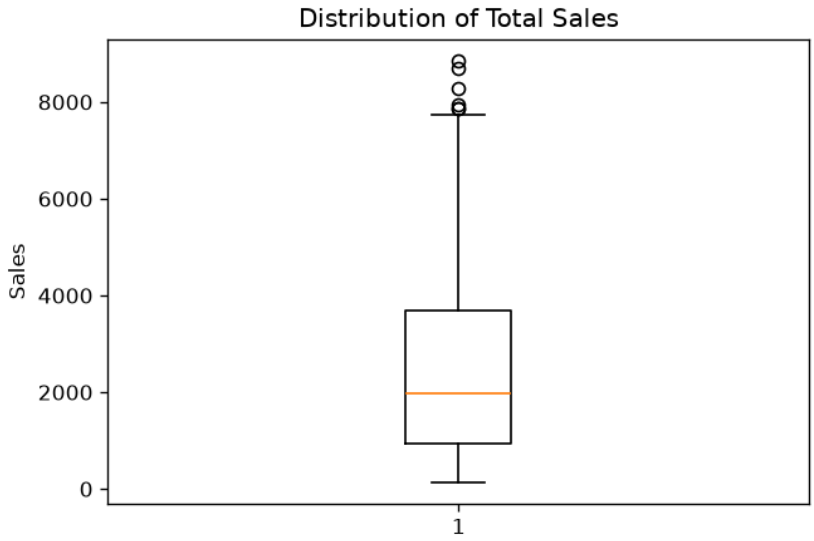
In [9]:

```
plt.figure(figsize=(6, 4))

plt.boxplot(df["Total_Sales"])

plt.title("Distribution of Total Sales")
plt.ylabel("Sales")
plt.show()
```

Plot [9]:



9. Save the cleaned data

In [10]:

```
df.to_csv("Cleaned_Sales_Data.csv", index=False)

print("Cleaned dataset saved successfully.")
```

Output [10]:

```
Cleaned dataset saved successfully.
```

Conclusion

- Sales data was loaded from CSV, Excel and JSON.
- The data was explored and cleaned.
- Total and average sales were calculated.
- Sales were grouped by region and product.
- Bar, pie and box plots were created.

- The cleaned dataset was exported.

Data Cleaning and Preparation

Lab 3 - Customer Churn

Dataset: A simple Telecom Customer Churn dataset generated using `sklearn.datasets.make_classification`.

Aim

To clean and prepare customer data for further churn analysis.

Steps

1. Load the dataset.
2. Explore the data.
3. Handle missing values.
4. Remove duplicates.
5. Standardize text values.
6. Convert data types.
7. Handle simple outliers.
8. Create a new feature.
9. Normalize numerical data.
10. Split into training and testing data.
11. Export the cleaned dataset.

In [1]:

```
import pandas as pd
import numpy as np

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

pd.set_option("display.max_columns", None)

# Ensure display works in all notebook environments
try:
    from IPython.display import display
except ImportError:
    pass

print("Libraries imported successfully.")
```

Output [1]:

```
Libraries imported successfully.
```

1. Create and load the dataset

`make_classification()` is part of `sklearn.datasets`. It creates a simple classification dataset that we use here as a small telecom churn dataset.

In [2]:

```
# Create a simple telecom customer dataset using sklearn
X, y = make_classification(
    n_samples=200,
    n_features=4,
    n_informative=3,
    n_redundant=0,
    random_state=42
)

df = pd.DataFrame(X, columns=[
    "Monthly_Charges",
    "Tenure_Months",
    "Support_Calls",
    "Data_Usage"
])

# Make values easier to understand
df["Monthly_Charges"] = (df["Monthly_Charges"] * 20 + 70).round(2)
df["Tenure_Months"] = (df["Tenure_Months"] * 10 + 30).round().astype(int)
df["Support_Calls"] = np.abs(df["Support_Calls"] * 2 + 3).round().astype(int)
df["Data_Usage"] = np.abs(df["Data_Usage"] * 5 + 10).round(2)

df["Contract"] = np.where(
```

```
    df["Tenure_Months"] > 36,
    "Long Term",
    "Month-to-Month"
)

df["Churn"] = np.where(y == 1, "Yes", "No")

# Add a few realistic data-quality problems
df.loc[5, "Monthly_Charges"] = np.nan
df.loc[15, "Contract"] = " month-to-month "
df.loc[25, "Contract"] = "LONG TERM"
df.loc[35, "Support_Calls"] = 100

# Add one duplicate row
df = pd.concat([df, df.iloc[[10]]], ignore_index=True)

df.to_csv("Telecom_Customer_Churn.csv", index=False)

print("Dataset created successfully.")
display(df.head())
```

Output [2]:

```
Dataset created successfully.
```

Table [2]:

	Monthly_Charges	Tenure_Months	Support_Calls	Data_Usage	Contract	Churn
0	74.71	37	3	13.08	Long Term	Yes
1	81.60	40	6	8.18	Long Term	Yes
2	59.58	34	5	10.85	Month-to-Month	Yes
3	43.87	49	0	4.45	Long Term	No
4	63.64	25	2	15.15	Month-to-Month	No

2. Explore the dataset

In [3]:

```
# Shape of the dataset
print("Rows and columns:", df.shape)

# Column names and data types
print("\nData types:")
print(df.dtypes)

# First few records
print("\nFirst 5 records:")
display(df.head())

# Basic statistics
print("\nBasic statistics:")
display(df.describe())
```

Output [3]:

```
Rows and columns: (201, 6)

Data types:
Monthly_Charges    float64
Tenure_Months        int64
Support_Calls        int64
Data_Usage         float64
Contract               str
Churn                  str
dtype: object

First 5 records:

Basic statistics:
```

Table [3]:

	Monthly_Charges	Tenure_Months	Support_Calls	Data_Usage	Contract	Churn
0	74.71	37	3	13.08	Long Term	Yes
1	81.60	40	6	8.18	Long Term	Yes
2	59.58	34	5	10.85	Month-to-Month	Yes
3	43.87	49	0	4.45	Long Term	No
4	63.64	25	2	15.15	Month-to-Month	No

Table [3]:

	Monthly_Charges	Tenure_Months	Support_Calls	Data_Usage
count	200.000000	201.000000	201.000000	201.000000
mean	63.024050	35.109453	3.427861	10.039104
std	24.943834	13.490291	7.337985	4.806134
min	2.450000	0.000000	0.000000	0.370000
25%	47.482500	26.000000	1.000000	7.120000
50%	62.445000	36.000000	2.000000	9.830000
75%	76.527500	46.000000	5.000000	13.160000
max	135.850000	65.000000	100.000000	23.160000

3. Check and handle missing values

In [4]:

```
print("Missing values before cleaning:")
print(df.isnull().sum())

# Fill missing Monthly_Charges with the median
df["Monthly_Charges"] = df["Monthly_Charges"].fillna(
    df["Monthly_Charges"].median()
)

print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

Output [4]:

```
Missing values before cleaning:
Monthly_Charges    1
Tenure_Months      0
Support_Calls      0
Data_Usage         0
Contract           0
Churn              0
dtype: int64

Missing values after cleaning:
Monthly_Charges    0
Tenure_Months      0
Support_Calls      0
Data_Usage         0
Contract           0
Churn              0
dtype: int64
```

4. Remove duplicate records

In [5]:

```
print("Duplicate rows before:", df.duplicated().sum())

df = df.drop_duplicates().reset_index(drop=True)

print("Duplicate rows after:", df.duplicated().sum())
```

Output [5]:

```
Duplicate rows before: 1
Duplicate rows after: 0
```

5. Standardize inconsistent text

In [6]:

```
# Remove extra spaces and standardize the Contract column
df["Contract"] = df["Contract"].str.strip().str.title()

print(df["Contract"].value_counts())
```

Output [6]:

```
Contract
Month-To-Month    103
Long Term          97
Name: count, dtype: int64
```

6. Convert columns to correct data types

In [7]:

```
df["Monthly_Charges"] = pd.to_numeric(df["Monthly_Charges"])
df["Tenure_Months"] = pd.to_numeric(df["Tenure_Months"], downcast="integer")
df["Support_Calls"] = pd.to_numeric(df["Support_Calls"], downcast="integer")
df["Data_Usage"] = pd.to_numeric(df["Data_Usage"])

print(df.dtypes)
```

Output [7]:

```
Monthly_Charges    float64
Tenure_Months         int8
Support_Calls         int8
Data_Usage         float64
Contract               str
Churn                  str
dtype: object
```

7. Handle a simple outlier

In [8]:

```
# Support calls should not contain an extremely large value.
# Replace values above 20 with 20.
df["Support_Calls"] = df["Support_Calls"].clip(upper=20)

print("Maximum Support Calls:", df["Support_Calls"].max())
```

Output [8]:

```
Maximum Support Calls: 20
```

8. Feature engineering

In [9]:

```
# Create a new feature: estimated yearly charges
df["Yearly_Charges"] = (df["Monthly_Charges"] * 12).round(2)

display(df.head())
```

Table [9]:

	Monthly_Charges	Tenure_Months	Support_Calls	Data_Usage	Contract	Churn	Yearly_Charges
0	74.71	37	3	13.08	Long Term	Yes	896.52
1	81.60	40	6	8.18	Long Term	Yes	979.20
2	59.58	34	5	10.85	Month-To-Month	Yes	714.96
3	43.87	49	0	4.45	Long Term	No	526.44
4	63.64	25	2	15.15	Month-To-Month	No	763.68

9. Normalize numerical data

In [10]:

```
numeric_columns = [
    "Monthly_Charges",
    "Tenure_Months",
    "Support_Calls",
    "Data_Usage",
    "Yearly_Charges"
]
```

```
scaler = MinMaxScaler()
df[numeric_columns] = scaler.fit_transform(df[numeric_columns])

display(df.head())
```

Table [10]:

	Monthly_Charges	Tenure_Months	Support_Calls	Data_Usage	Contract	Churn	Yearly_Charges
0	0.541679	0.569231	0.15	0.557701	Long Term	Yes	0.541679
1	0.593328	0.615385	0.30	0.342694	Long Term	Yes	0.593328
2	0.428261	0.523077	0.25	0.459851	Month-To-Month	Yes	0.428261
3	0.310495	0.753846	0.00	0.179026	Long Term	No	0.310495
4	0.458696	0.384615	0.10	0.648530	Month-To-Month	No	0.458696

10. Split the data into training and testing sets

In [11]:

```
# Separate features and target
X = df.drop("Churn", axis=1)
y = df["Churn"]

# Convert categorical Contract column into numeric columns
X = pd.get_dummies(X, columns=["Contract"], drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data :", X_test.shape)
```

Output [11]:

```
Training data: (160, 6)
Testing data : (40, 6)
```

11. Export the cleaned dataset

In [12]:

```
df.to_csv("Cleaned_Telecom_Customer_Churn.csv", index=False)

print("Cleaned dataset exported successfully.")
display(df.head())
```

Output [12]:

```
Cleaned dataset exported successfully.
```

Table [12]:

	Monthly_Charges	Tenure_Months	Support_Calls	Data_Usage	Contract	Churn	Yearly_Charges
0	0.541679	0.569231	0.15	0.557701	Long Term	Yes	0.541679
1	0.593328	0.615385	0.30	0.342694	Long Term	Yes	0.593328
2	0.428261	0.523077	0.25	0.459851	Month-To-Month	Yes	0.428261
3	0.310495	0.753846	0.00	0.179026	Long Term	No	0.310495
4	0.458696	0.384615	0.10	0.648530	Month-To-Month	No	0.458696

Conclusion

The telecom customer data was cleaned and prepared by: - Handling missing values - Removing duplicate records - Standardizing text - Correcting data types - Handling a simple outlier - Creating a new feature - Normalizing numerical columns - Splitting the data into training and testing sets - Exporting the cleaned dataset

Data Wrangling

Lab 4 - Real Estate Market

Dataset: California Housing dataset from `sklearn.datasets`

Aim

To perform simple data wrangling on housing-price data.

Tasks

1. Load the dataset.
2. Clean column names.
3. Handle missing values.
4. Merge an additional small dataset.
5. Filter the data.
6. Encode categorical data.
7. Calculate summary statistics.
8. Handle simple outliers.

In [1]:

```
import pandas as pd
import numpy as np

from sklearn.datasets import fetch_california_housing

pd.set_option("display.max_columns", None)

# Ensure display works in all notebook environments
try:
    from IPython.display import display
except ImportError:
    pass

print("Libraries imported successfully.")
```

Output [1]:

```
Libraries imported successfully.
```

1. Load the dataset

The **California Housing dataset** is available through `sklearn.datasets`.

In [2]:

```
# Load the California Housing dataset
housing = fetch_california_housing(as_frame=True)

df = housing.frame.copy()

print("Dataset shape:", df.shape)
display(df.head())
```

Output [2]:

```
Dataset shape: (20640, 9)
```

Table [2]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

2. Clean the column names

In [3]:

```
# Make column names easier to use
df.columns = (
    df.columns
    .str.strip()
```

```
    .str.replace(" ", "_")
    .str.lower()
)

# Rename the target column
df = df.rename(columns={"medhouseval": "house_price"})

# Actual sklearn column names are kept in lowercase (e.g. averooms, medinc).

print("Clean column names:")
print(df.columns.tolist())
```

Output [3]:

```
Clean column names:
['medinc', 'houseage', 'averooms', 'avebedrms', 'population', 'aveoccup', 'latitude', 'longitude', 'house_price']
```

3. Check and handle missing values

In [4]:

```
print("Missing values before cleaning:")
print(df.isnull().sum())

# Fill missing numerical values with the median
df = df.fillna(df.median(numeric_only=True))

print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

Output [4]:

```
Missing values before cleaning:
medinc         0
houseage       0
averooms       0
avebedrms      0
population     0
aveoccup       0
latitude       0
longitude      0
house_price    0
dtype: int64

Missing values after cleaning:
medinc         0
houseage       0
averooms       0
avebedrms      0
population     0
aveoccup       0
latitude       0
longitude      0
house_price    0
dtype: int64
```

4. Add a simple categorical column

The original dataset mainly contains numerical information. We create a simple **property type** column so that categorical-data handling can be demonstrated.

In [5]:

```
# Create a simple property type
# If a house has more than 5 average rooms, call it "Large".
df["property_type"] = np.where(
    df["averooms"] > 5,
    "Large",
    "Small"
)

display(df[["averooms", "property_type"]].head())
```

Table [5]:

	averooms	property_type
0	6.984127	Large
1	6.238137	Large
2	8.288136	Large
3	5.817352	Large
4	6.281853	Large

5. Merge additional information

In [6]:

```
# Create a small additional dataset based on the income level
# This represents simple neighborhood information.
neighborhood_info = pd.DataFrame({
    "income_group": ["Low", "Medium", "High"],
    "amenity_score": [4, 7, 9]
})

# Create the same income group in the housing data
df["income_group"] = pd.cut(
    df["medinc"],
    bins=[0, 2.5, 5, np.inf],
    labels=["Low", "Medium", "High"]
)

# Merge the two datasets
df = df.merge(
    neighborhood_info,
    on="income_group",
    how="left"
)

display(df.head())
```

Table [6]:

	medinc	houseage	averooms	avebedrms	population	aveoccup	latitude	longitude	house_price	property_type	income_group	amer
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526	Large	High	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585	Large	High	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521	Large	High	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413	Large	High	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422	Large	Medium	

6. Filter the data

In [7]:

```
# Select houses with:
# - more than 5 rooms
# - house price greater than 2
filtered_df = df[
    (df["averooms"] > 5) &
    (df["house_price"] > 2)
]

print("Filtered data:")
display(filtered_df.head())

print("Number of filtered records:", len(filtered_df))
```

Output [7]:

```
Filtered data:
Number of filtered records: 5978
```

Table [7]:

	medinc	houseage	averooms	avebedrms	population	aveoccup	latitude	longitude	house_price	property_type	income_group	ame
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526	Large	High	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585	Large	High	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521	Large	High	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413	Large	High	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422	Large	Medium	

7. Encode categorical variables

In [8]:

```
# One-hot encode the property_type column
df_encoded = pd.get_dummies(
    df,
    columns=["property_type"],
    drop_first=True
)

display(df_encoded.head())
```

Table [8]:

	medinc	houseage	averooms	avebedrms	population	aveoccup	latitude	longitude	house_price	income_group	amenity_score	prop
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526	High	9	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585	High	9	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521	High	9	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413	High	9	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422	Medium	7	

8. Calculate summary statistics

In [9]:

```
# Average house price by property type
average_price = (
    df.groupby("property_type")["house_price"]
    .mean()
)

print("Average house price by property type:")
print(average_price)

print("\nOverall summary:")
display(
    df[["medinc", "averooms", "house_price"]].describe()
)
```

Output [9]:

```
Average house price by property type:
property_type
Large    2.264487
Small    1.801260
Name: house_price, dtype: float64

Overall summary:
```

Table [9]:

	medinc	averooms	house_price
count	20640.000000	20640.000000	20640.000000
mean	3.870671	5.429000	2.068558
std	1.899822	2.474173	1.153956
min	0.499900	0.846154	0.149990
25%	2.563400	4.440716	1.196000
50%	3.534800	5.229129	1.797000
75%	4.743250	6.052381	2.647250
max	15.000100	141.909091	5.000010

9. Handle outliers

In [10]:

```
# Simple outlier handling using the IQR method
Q1 = df["house_price"].quantile(0.25)
Q3 = df["house_price"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

# Remove values outside the limits
clean_df = df[
    (df["house_price"] >= lower_limit) &
    (df["house_price"] <= upper_limit)
].copy()

print("Rows before removing outliers:", len(df))
print("Rows after removing outliers :", len(clean_df))
```

Output [10]:

```
Rows before removing outliers: 20640
Rows after removing outliers : 19569
```

10. Final cleaned dataset

In [11]:

```
print("Final dataset shape:", clean_df.shape)
display(clean_df.head())

# Save the cleaned dataset
clean_df.to_csv("Cleaned_RealEstate_Prices.csv", index=False)

print("Cleaned dataset exported successfully.")
```

Output [11]:

```
Final dataset shape: (19569, 12)
Cleaned dataset exported successfully.
```

Table [11]:

	medinc	houseage	averooms	avebedrms	population	aveoccup	latitude	longitude	house_price	property_type	income_group	amer
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526	Large	High	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585	Large	High	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521	Large	High	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413	Large	High	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422	Large	Medium	

Conclusion

The real estate data was wrangled by: - Loading the California Housing dataset from sklearn - Cleaning column names - Handling missing values - Adding and merging neighborhood information - Filtering records - Encoding a categorical variable - Calculating average house prices - Detecting and removing simple outliers - Exporting the cleaned dataset

Data Visualization using Matplotlib

Lab 5 - Air Quality Index (AQI)

Aim

To visualize AQI trends and pollutant levels using **Matplotlib**.

Dataset note

`sklearn.datasets` does not provide a dedicated real-world AQI dataset. Therefore, this lab creates a **simple AQI-style dataset using `sklearn.datasets.make_regression`** and transforms it into pollutant values such as PM2.5, PM10 and CO.

This keeps the lab fully based on the sklearn library while making the visualization easy to understand.

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_regression

# Create sample data using sklearn
X, y = make_regression(
    n_samples=30,
    n_features=3,
    random_state=42
)

# Convert the generated values into simple pollutant ranges
df = pd.DataFrame(X, columns=["PM25_raw", "PM10_raw", "CO_raw"])

df["Date"] = pd.date_range("2025-01-01", periods=30)

df["PM25"] = (df["PM25_raw"] - df["PM25_raw"].min()) / (
    df["PM25_raw"].max() - df["PM25_raw"].min()
) * 150 + 20

df["PM10"] = (df["PM10_raw"] - df["PM10_raw"].min()) / (
    df["PM10_raw"].max() - df["PM10_raw"].min()
) * 200 + 30

df["CO"] = (df["CO_raw"] - df["CO_raw"].min()) / (
    df["CO_raw"].max() - df["CO_raw"].min()
) * 10 + 1

# Create a simple AQI value from pollutant levels
df["AQI"] = (
    0.45 * df["PM25"] +
    0.35 * df["PM10"] +
    5 * df["CO"]
)

df["AQI"] = df["AQI"].round(2)

# Keep only the columns needed for the lab
df = df[["Date", "PM25", "PM10", "CO", "AQI"]]

# Save the dataset
df.to_csv("City_Air_Quality.csv", index=False)

print("Dataset created successfully.")
display(df.head())
```

Output [1]:

```
Dataset created successfully.
```

Table [1]:

	Date	PM25	PM10	CO	AQI
0	2025-01-01	127.559080	111.010927	8.789638	140.20
1	2025-01-02	61.171827	107.866309	5.392011	92.24
2	2025-01-03	170.000000	69.711970	7.117339	136.49
3	2025-01-04	106.065419	225.951390	5.566368	154.64
4	2025-01-05	70.468070	39.980163	8.318296	87.30

1. Explore the dataset

In [2]:

```
print("Dataset shape:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nFirst 5 rows:")
display(df.head())

print("\nBasic statistics:")
display(df.describe())
```

Output [2]:

```
Dataset shape: (30, 5)

Column names:
['Date', 'PM25', 'PM10', 'CO', 'AQI']

Data types:
Date    datetime64[us]
PM25           float64
PM10           float64
CO             float64
AQI            float64
dtype: object

First 5 rows:

Basic statistics:
```

Table [2]:

	Date	PM25	PM10	CO	AQI
0	2025-01-01	127.559080	111.010927	8.789638	140.20
1	2025-01-02	61.171827	107.866309	5.392011	92.24
2	2025-01-03	170.000000	69.711970	7.117339	136.49
3	2025-01-04	106.065419	225.951390	5.566368	154.64
4	2025-01-05	70.468070	39.980163	8.318296	87.30

Table [2]:

	Date	PM25	PM10	CO	AQI
count	30	30.000000	30.000000	30.000000	30.000000
mean	2025-01-15 12:00:00	108.315514	114.933084	5.897314	118.455000
min	2025-01-01 00:00:00	20.000000	30.000000	1.000000	76.270000
25%	2025-01-08 06:00:00	77.008386	76.003518	4.658088	96.967500
50%	2025-01-15 12:00:00	106.326347	112.314660	5.754375	118.045000
75%	2025-01-22 18:00:00	135.692192	141.690911	8.155914	139.190000
max	2025-01-30 00:00:00	170.000000	230.000000	11.000000	165.280000
std	NaN	37.505241	50.441225	2.730826	23.756602

2. Identify variables for visualization

- **Date** → time
- **PM25** → PM2.5 pollutant level
- **PM10** → PM10 pollutant level

- **CO** → carbon monoxide level
- **AQI** → overall air quality index

3. Line plot - Overall AQI trend

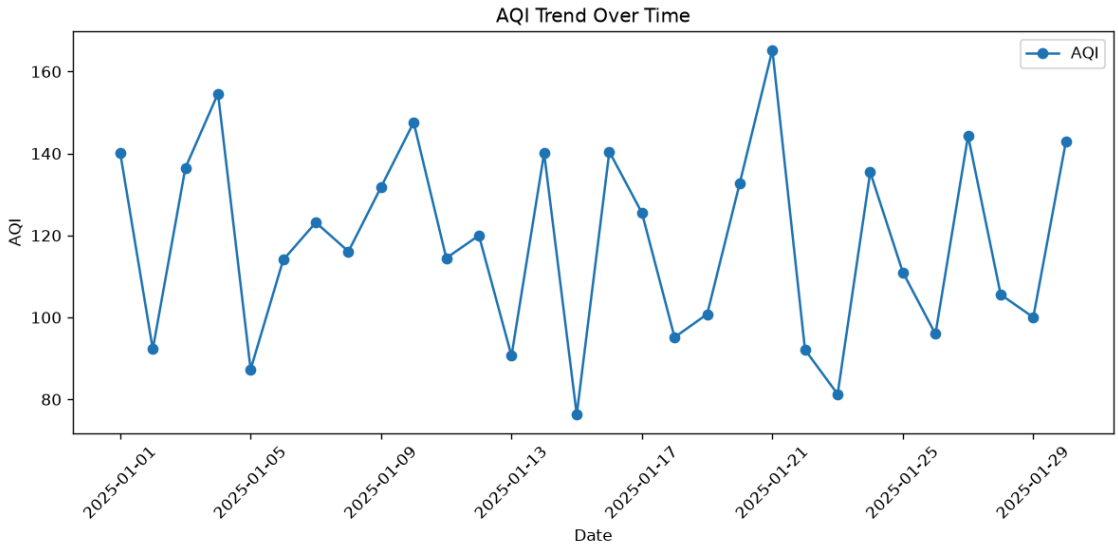
In [3]:

```
plt.figure(figsize=(10, 5))

plt.plot(df["Date"], df["AQI"], marker="o", label="AQI")

plt.title("AQI Trend Over Time")
plt.xlabel("Date")
plt.ylabel("AQI")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```

Plot [3]:



4. Line plots - Individual pollutants

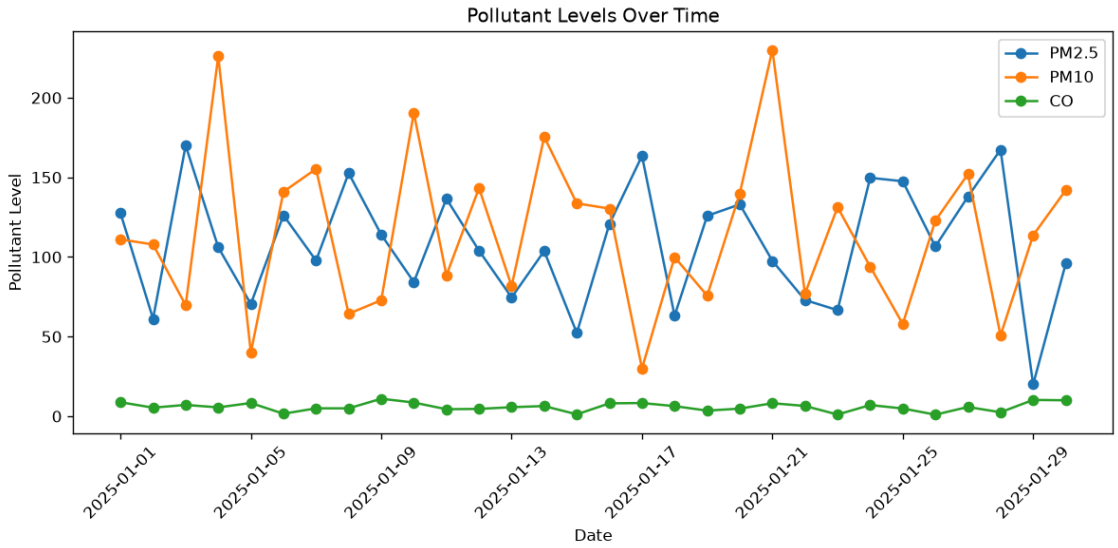
In [4]:

```
plt.figure(figsize=(10, 5))

plt.plot(df["Date"], df["PM25"], marker="o", label="PM2.5")
plt.plot(df["Date"], df["PM10"], marker="o", label="PM10")
plt.plot(df["Date"], df["CO"], marker="o", label="CO")

plt.title("Pollutant Levels Over Time")
plt.xlabel("Date")
plt.ylabel("Pollutant Level")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```

Plot [4]:



5. Bar plot - AQI values

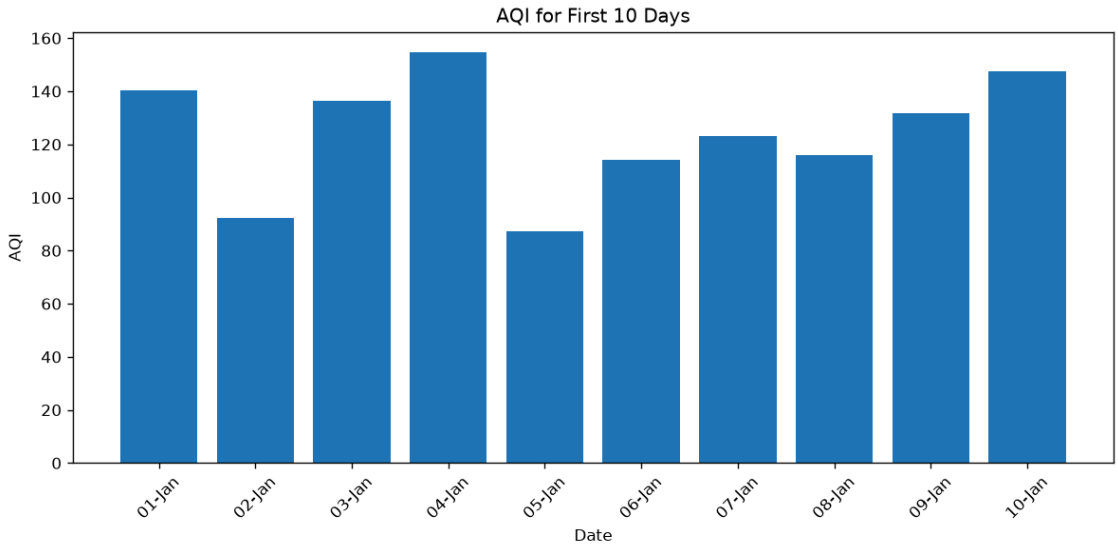
In [5]:

```
# Show AQI for the first 10 days
sample = df.head(10)

plt.figure(figsize=(10, 5))
plt.bar(sample["Date"].dt.strftime("%d-%b"), sample["AQI"])

plt.title("AQI for First 10 Days")
plt.xlabel("Date")
plt.ylabel("AQI")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Plot [5]:



6. Box plot - Distribution of pollutants

In [6]:

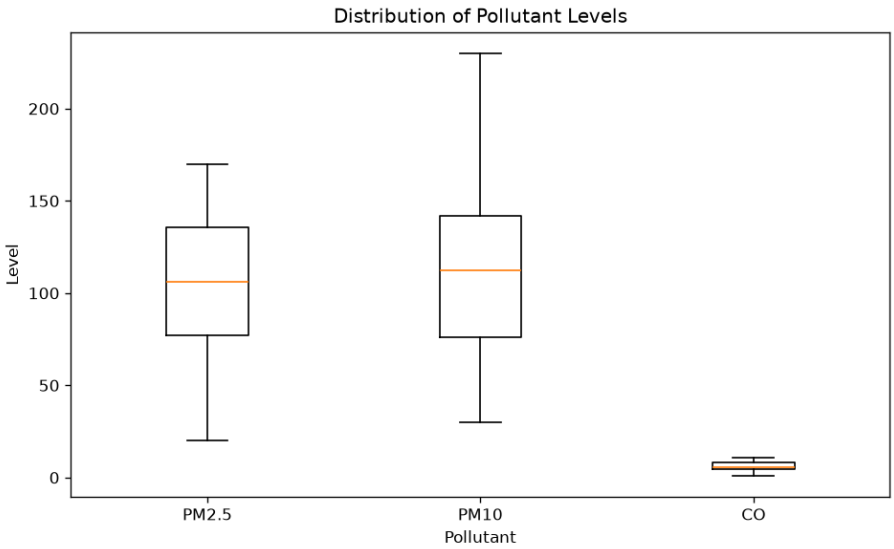
```
plt.figure(figsize=(8, 5))

plt.boxplot(
    [df["PM25"], df["PM10"], df["CO"]],
    tick_labels=["PM2.5", "PM10", "CO"]
)

plt.title("Distribution of Pollutant Levels")
plt.xlabel("Pollutant")
plt.ylabel("Level")
```

```
plt.tight_layout()
plt.show()
```

Plot [6]:



7. Scatter plot - AQI and PM2.5

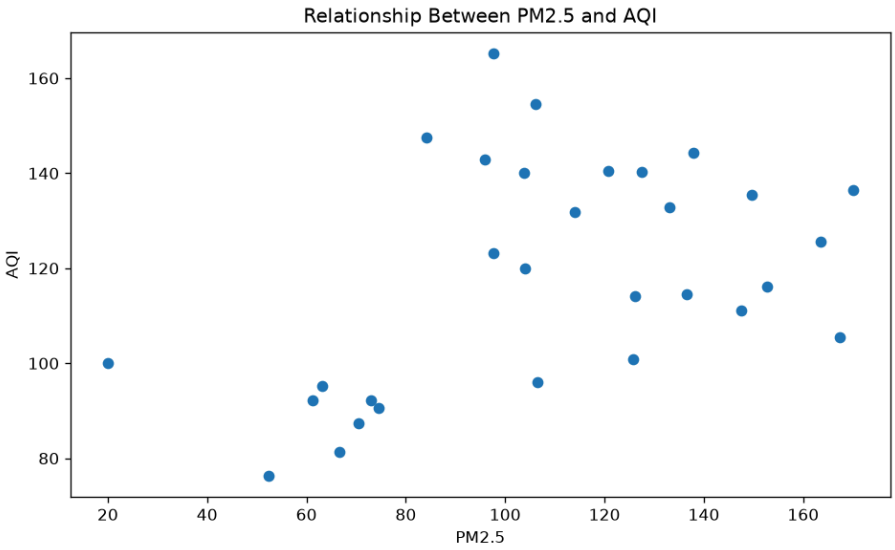
In [7]:

```
plt.figure(figsize=(8, 5))

plt.scatter(df["PM25"], df["AQI"])

plt.title("Relationship Between PM2.5 and AQI")
plt.xlabel("PM2.5")
plt.ylabel("AQI")
plt.tight_layout()
plt.show()
```

Plot [7]:



8. Scatter plot - AQI and PM10

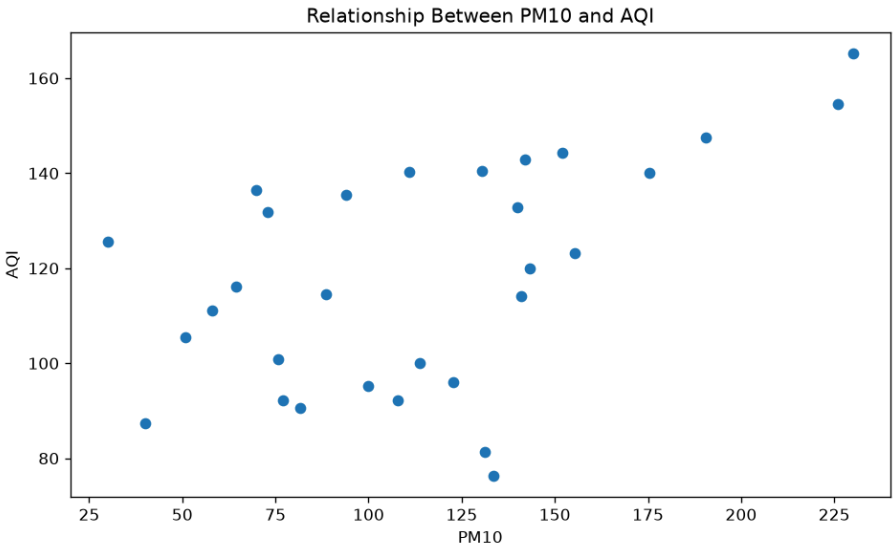
In [8]:

```
plt.figure(figsize=(8, 5))

plt.scatter(df["PM10"], df["AQI"])

plt.title("Relationship Between PM10 and AQI")
plt.xlabel("PM10")
plt.ylabel("AQI")
plt.tight_layout()
plt.show()
```

Plot [8]:



9. Add labels and find the highest AQI

In [9]:

```
highest_aqi = df.loc[df["AQI"].idxmax()]

print("Highest AQI:")
print(highest_aqi)

plt.figure(figsize=(10, 5))

plt.plot(df["Date"], df["AQI"], marker="o")

plt.title("AQI Trend with Labels")
plt.xlabel("Date")
plt.ylabel("AQI")
plt.xticks(rotation=45)

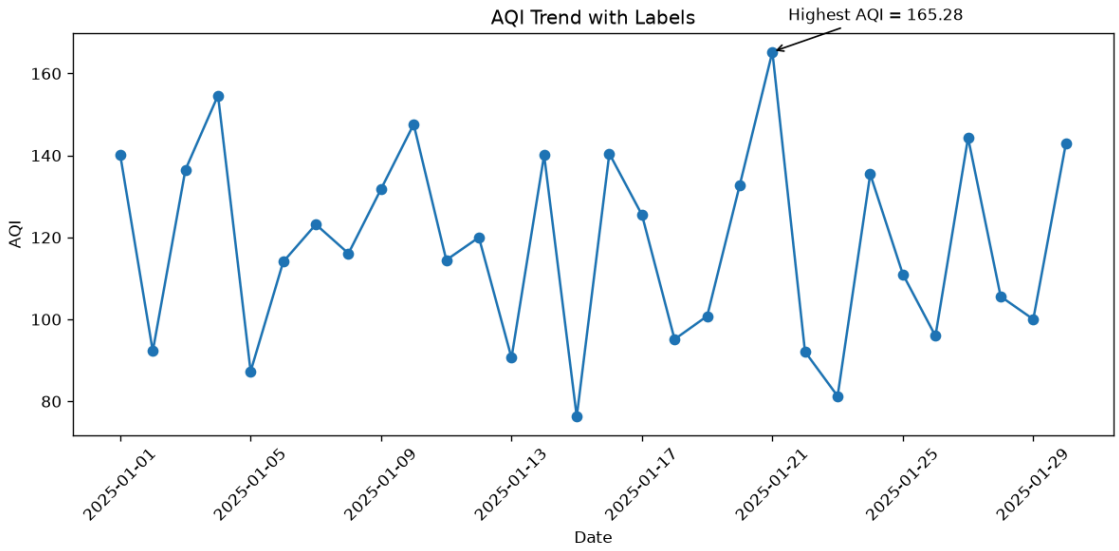
# Mark the highest AQI
plt.annotate(
    f"Highest AQI = {highest_aqi['AQI']}",
    xy=(highest_aqi["Date"], highest_aqi["AQI"]),
    xytext=(10, 20),
    textcoords="offset points",
    arrowprops=dict(arrowstyle="->")
)

plt.tight_layout()
plt.show()
```

Output [9]:

```
Highest AQI:
Date    2025-01-21 00:00:00
PM25              97.575167
PM10                  230.0
CO                 8.174774
AQI                  165.28
Name: 20, dtype: object
```

Plot [9]:



Conclusion

The AQI-style dataset was visualized using Matplotlib.

The following plots were created: - Line plot for AQI trend - Line plot for pollutant levels - Bar plot for AQI comparison - Box plot for pollutant distribution - Scatter plots for AQI and pollutant relationships

Labels, titles, legends and axis names were also added to make the graphs easy to understand.

Data Aggregation

Lab 6 - Retail Sales Analysis by Region

Aim

To aggregate retail sales data by **region** and **product category** and identify the regions with the highest sales.

Dataset note

A dedicated retail sales dataset is not available in `sklearn.datasets`. Therefore, the **Wine dataset from sklearn** is used as the base, and simple retail-sales columns are created from it for this lab.

Tasks

1. Load and explore the dataset.
2. Identify variables used for aggregation.
3. Calculate total sales by region.
4. Visualize sales using bar and pie charts.
5. Find the top-performing region.
6. Calculate sales by region and product category.
7. Create a grouped bar chart.

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine

pd.set_option("display.max_columns", None)

# Ensure display works in all notebook environments
try:
    from IPython.display import display
except ImportError:
    pass

print("Libraries imported successfully.")
```

Output [1]:

```
Libraries imported successfully.
```

1. Create the retail sales dataset using sklearn

In [2]:

```
# Load the Wine dataset from sklearn
wine = load_wine(as_frame=True)
wine_df = wine.frame

# Create a simple retail sales dataset
np.random.seed(42)

df = pd.DataFrame({
    "Transaction_Date": pd.date_range(
        "2025-01-01", periods=len(wine_df), freq="D"
    ),
    "Region": np.random.choice(
        ["North", "South", "East", "West"],
        size=len(wine_df)
    ),
    "Product_Category": wine_df["target"].map({
        0: "Electronics",
        1: "Clothing",
        2: "Grocery"
    }),
    "Quantity_Sold": np.random.randint(1, 11, size=len(wine_df)),
    "Sales_Amount": np.round(
        np.random.uniform(500, 5000, size=len(wine_df)), 2
    )
})

# Save the dataset
df.to_csv("Retail_Sales_Data.csv", index=False)
```

```
print("Dataset created successfully.")
display(df.head())
```

Output [2]:

```
Dataset created successfully.
```

Table [2]:

	Transaction_Date	Region	Product_Category	Quantity_Sold	Sales_Amount
0	2025-01-01	East	Electronics	1	2761.87
1	2025-01-02	West	Electronics	3	3096.07
2	2025-01-03	North	Electronics	5	2716.33
3	2025-01-04	East	Electronics	3	1378.59
4	2025-01-05	East	Electronics	1	3751.03

2. Explore the dataset

In [3]:

```
print("Dataset shape:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nFirst 5 records:")
display(df.head())

print("\nBasic statistics:")
display(df.describe())
```

Output [3]:

```
Dataset shape: (178, 5)

Column names:
['Transaction_Date', 'Region', 'Product_Category', 'Quantity_Sold', 'Sales_Amount']

Data types:
Transaction_Date    datetime64[us]
Region                         str
Product_Category               str
Quantity_Sold                int32
Sales_Amount               float64
dtype: object

First 5 records:

Basic statistics:
```

Table [3]:

	Transaction_Date	Region	Product_Category	Quantity_Sold	Sales_Amount
0	2025-01-01	East	Electronics	1	2761.87
1	2025-01-02	West	Electronics	3	3096.07
2	2025-01-03	North	Electronics	5	2716.33
3	2025-01-04	East	Electronics	3	1378.59
4	2025-01-05	East	Electronics	1	3751.03

Table [3]:

	Transaction_Date	Quantity_Sold	Sales_Amount
count	178	178.000000	178.000000
mean	2025-03-30 12:00:00	5.140449	2797.602360
min	2025-01-01 00:00:00	1.000000	548.770000
25%	2025-02-14 06:00:00	3.000000	1764.697500
50%	2025-03-30 12:00:00	5.000000	2870.595000
75%	2025-05-13 18:00:00	7.750000	3906.592500
max	2025-06-27 00:00:00	10.000000	4957.270000
std	NaN	2.855749	1332.552985

3. Relevant variables

The main variables used for aggregation are:

- **Region** → region of the sale
- **Product_Category** → type of product
- **Quantity_Sold** → number of products sold
- **Sales_Amount** → amount of the sale

4. Total sales by region

In [4]:

```
# Group sales by region
region_sales = (
    df.groupby("Region")["Sales_Amount"]
    .sum()
    .sort_values(ascending=False)
)

print("Total Sales by Region:")
display(region_sales)
```

Output [4]:

```
Total Sales by Region:
```

Output [4]:

```
Region
West     148374.57
East     118192.17
South    117197.23
North    114209.25
Name: Sales_Amount, dtype: float64
```

5. Bar chart - Sales by region

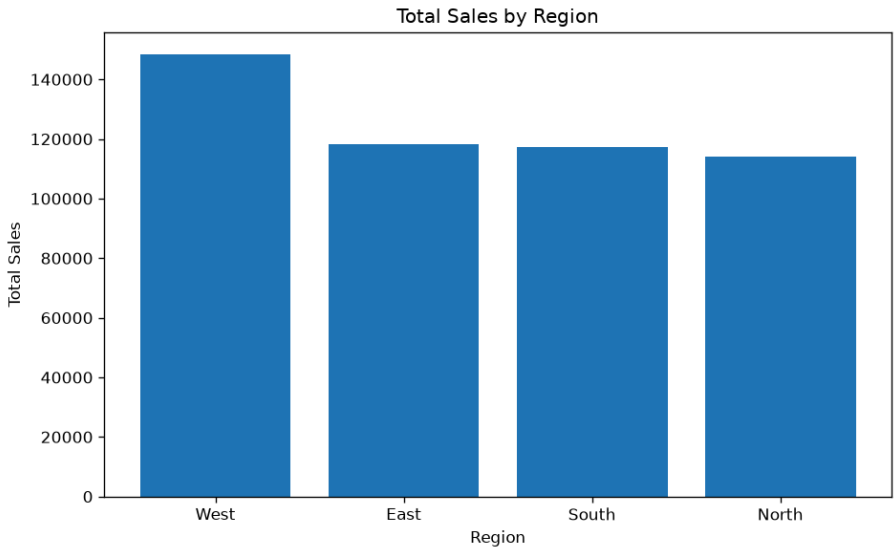
In [5]:

```
plt.figure(figsize=(8, 5))

plt.bar(region_sales.index, region_sales.values)

plt.title("Total Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.show()
```

Plot [5]:



6. Pie chart - Sales distribution by region

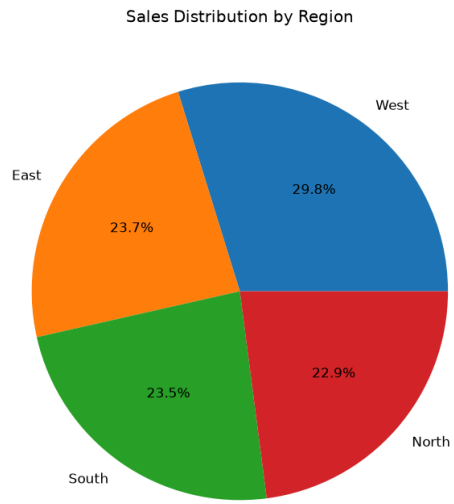
In [6]:

```
plt.figure(figsize=(7, 7))

plt.pie(
    region_sales.values,
    labels=region_sales.index,
    autopct="%1.1f%%"
)

plt.title("Sales Distribution by Region")
plt.show()
```

Plot [6]:



7. Identify the top-performing region

In [7]:

```
top_region = region_sales.idxmax()
top_sales = region_sales.max()

print("Top-performing region:", top_region)
print(f"Total sales in {top_region}: ₹{top_sales:,.2f}")
```

Output [7]:

```
Top-performing region: West
Total sales in West: ₹148,374.57
```

8. Sales by region and product category

In [8]:

```
region_category_sales = pd.pivot_table(
    df,
    values="Sales_Amount",
    index="Region",
    columns="Product_Category",
    aggfunc="sum",
    fill_value=0
)

print("Sales by Region and Product Category:")
display(region_category_sales)
```

Output [8]:

```
Sales by Region and Product Category:
```

Table [8]:

Product_Category	Clothing	Electronics	Grocery
Region			
East	28254.71	46271.05	43666.41
North	38584.45	28937.83	46686.97
South	59677.73	43968.69	13550.81
West	52347.03	60242.95	35784.59

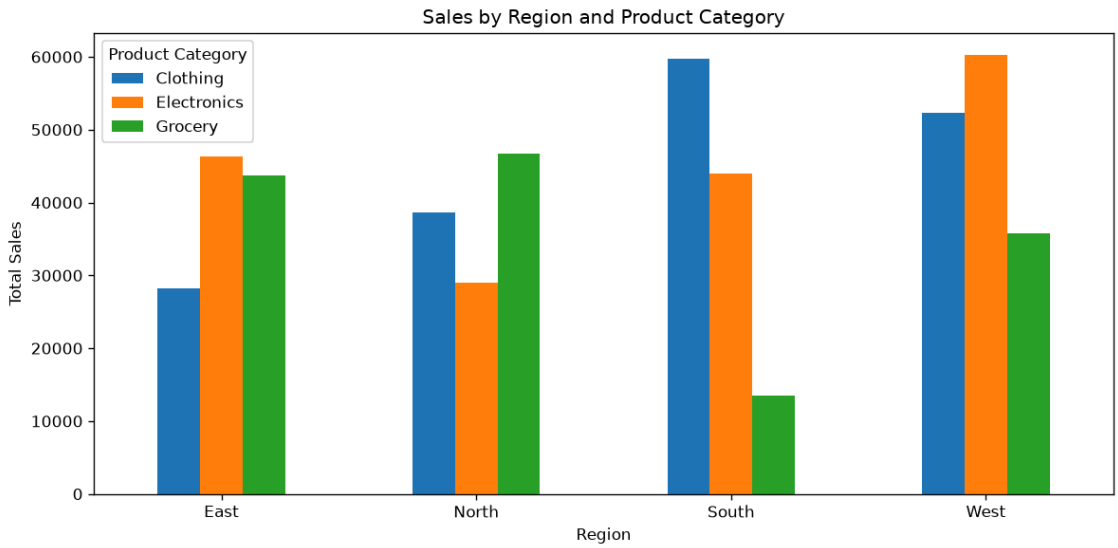
9. Grouped bar chart

In [9]:

```
region_category_sales.plot(
    kind="bar",
    figsize=(10, 5)
)

plt.title("Sales by Region and Product Category")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.xticks(rotation=0)
plt.legend(title="Product Category")
plt.tight_layout()
plt.show()
```

Plot [9]:



Conclusion

The retail sales data was analyzed using pandas aggregation functions.

- Sales were grouped by region.
- Total sales for each region were calculated.
- Bar and pie charts were created.
- The top-performing region was identified.

- Sales were also grouped by region and product category.
- A grouped bar chart was created to compare the combinations.

Time Series Data Analysis

Lab 7 - Stock Market Data

Aim

To analyze and visualize stock-price data using simple time-series techniques.

Dataset note

`sklearn.datasets` does not provide a stock-market dataset. Therefore, the **Diabetes dataset from sklearn** is used as the base data, and a simple stock-style dataset with **Date, Closing Price, and Volume** is created from it.

Tasks

1. Load and explore the dataset.
2. Convert the date column to datetime.
3. Plot stock-price trends.
4. Calculate a moving average.
5. Perform simple seasonal analysis.
6. Analyze correlation with trading volume.
7. Forecast future prices using exponential smoothing.

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from statsmodels.tsa.holtwinters import ExponentialSmoothing

pd.set_option("display.max_columns", None)

# Ensure display works in all notebook environments
try:
    from IPython.display import display
except ImportError:
    pass

print("Libraries imported successfully.")
```

Output [1]:

```
Libraries imported successfully.
```

1. Create the stock-price dataset using sklearn

In [2]:

```
# Load a dataset from sklearn
diabetes = load_diabetes(as_frame=True)

base = diabetes.frame.copy()

# Use the target values as the base for a simple stock-price series
np.random.seed(42)

price_base = (
    (base["target"] - base["target"].min()) /
    (base["target"].max() - base["target"].min())
)

# Create a simple stock-style closing price
closing_price = 100 + price_base * 80 + np.arange(len(base)) * 0.08
closing_price = closing_price + np.random.normal(0, 2, len(base))

# Create trading volume
volume = np.random.randint(1000, 10000, len(base))

df = pd.DataFrame({
    "Date": pd.date_range("2025-01-01", periods=len(base), freq="D"),
    "Closing_Price": np.round(closing_price, 2),
    "Volume": volume
})

# Save as CSV
```

```
df.to_csv("Stock_Prices.csv", index=False)

print("Dataset created successfully.")
display(df.head())
```

Output [2]:

```
Dataset created successfully.
```

Table [2]:

	Date	Closing_Price	Volume
0	2025-01-01	132.40	6569
1	2025-01-02	112.26	5300
2	2025-01-03	130.37	7269
3	2025-01-04	148.40	4818
4	2025-01-05	127.27	2906

2. Explore the dataset

In [3]:

```
print("Dataset shape:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nBasic statistics:")
display(df.describe())
```

Output [3]:

```
Dataset shape: (442, 3)

Column names:
['Date', 'Closing_Price', 'Volume']

Data types:
Date             datetime64[us]
Closing_Price           float64
Volume                    int32
dtype: object

Basic statistics:
```

Table [3]:

	Date	Closing_Price	Volume
count	442	442.000000	442.000000
mean	2025-08-09 12:00:00	149.376697	5546.314480
min	2025-01-01 00:00:00	106.930000	1005.000000
25%	2025-04-21 06:00:00	131.725000	3376.500000
50%	2025-08-09 12:00:00	145.495000	5507.500000
75%	2025-11-27 18:00:00	167.430000	7697.250000
max	2026-03-18 00:00:00	207.380000	9996.000000
std	NaN	22.464546	2535.959446

3. Convert Date column to datetime

In [4]:

```
df["Date"] = pd.to_datetime(df["Date"])

# Set Date as the index for time-series analysis
df = df.set_index("Date")

print(df.dtypes)
display(df.head())
```

Output [4]:

```
Closing_Price    float64
Volume             int32
dtype: object
```

Table [4]:

	Closing_Price	Volume
Date		
2025-01-01	132.40	6569
2025-01-02	112.26	5300
2025-01-03	130.37	7269
2025-01-04	148.40	4818
2025-01-05	127.27	2906

4. Plot historical stock-price trend

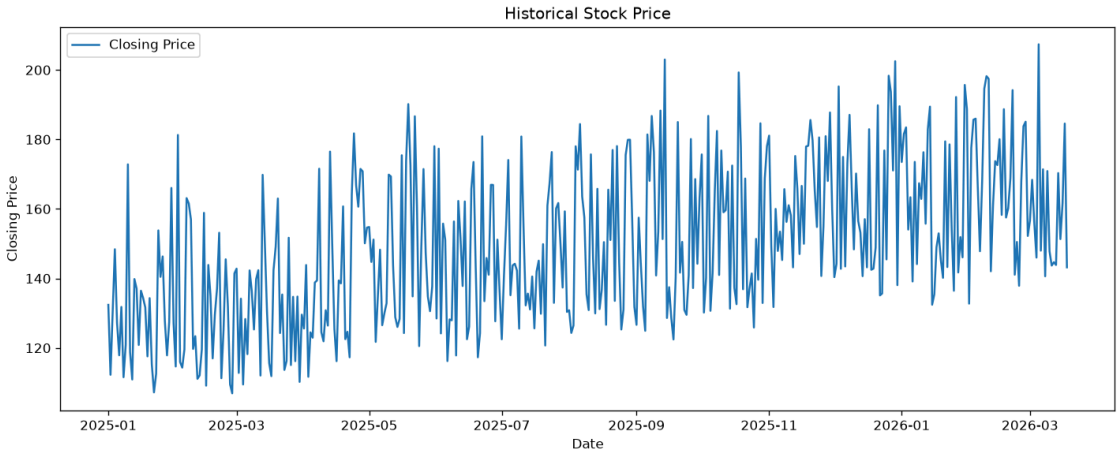
In [5]:

```
plt.figure(figsize=(12, 5))

plt.plot(df.index, df["Closing_Price"], label="Closing Price")

plt.title("Historical Stock Price")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.legend()
plt.tight_layout()
plt.show()
```

Plot [5]:



5. Calculate and plot moving average

In [6]:

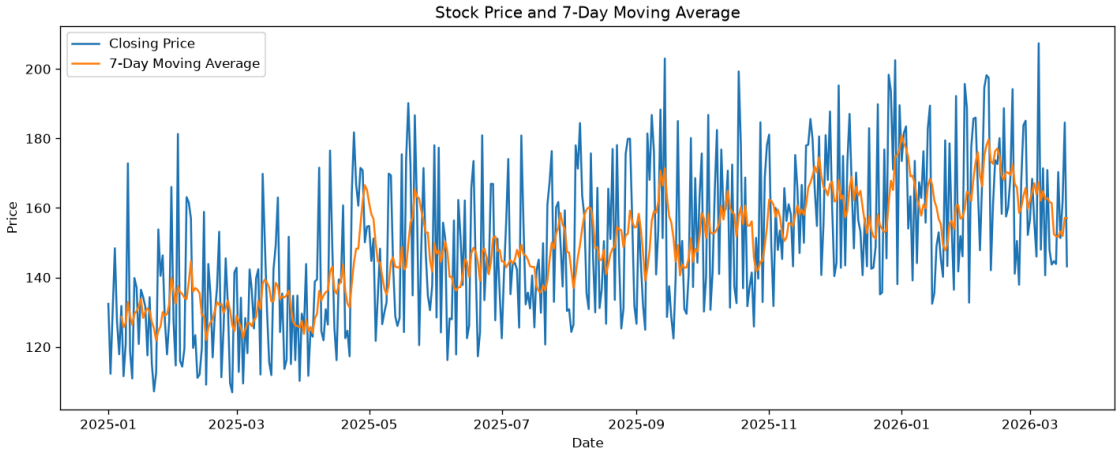
```
# Calculate a 7-day moving average
df["Moving_Average_7"] = df["Closing_Price"].rolling(window=7).mean()

plt.figure(figsize=(12, 5))

plt.plot(df.index, df["Closing_Price"], label="Closing Price")
plt.plot(df.index, df["Moving_Average_7"], label="7-Day Moving Average")

plt.title("Stock Price and 7-Day Moving Average")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.tight_layout()
plt.show()
```

Plot [6]:



6. Simple seasonality analysis

In [7]:

```
# Calculate average closing price for each month
monthly_average = df.groupby(df.index.month)["Closing_Price"].mean()

print("Average Closing Price by Month:")
display(monthly_average)

plt.figure(figsize=(8, 5))

plt.plot(
    monthly_average.index,
    monthly_average.values,
    marker="o"
)

plt.title("Average Stock Price by Month")
plt.xlabel("Month")
plt.ylabel("Average Closing Price")
plt.xticks(range(1, 13))
plt.tight_layout()
plt.show()
```

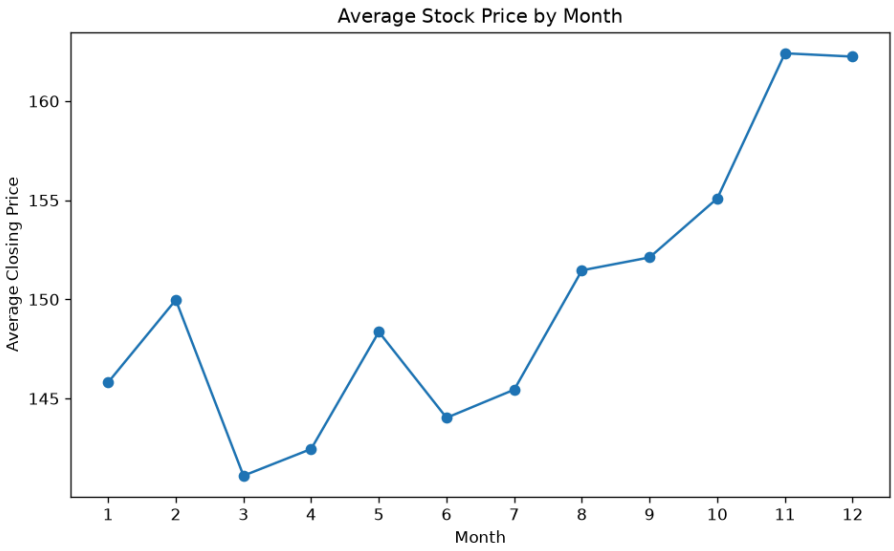
Output [7]:

```
Average Closing Price by Month:
```

Output [7]:

```
Date
1     145.811774
2     149.974286
3     141.105306
4     142.455000
5     148.360968
6     144.027667
7     145.446452
8     151.465161
9     152.118333
10    155.094194
11    162.414000
12    162.246129
Name: Closing_Price, dtype: float64
```

Plot [7]:



7. Correlation between stock price and trading volume

In [8]:

```
correlation = df["Closing_Price"].corr(df["Volume"])

print(f"Correlation between Closing Price and Volume: {correlation:.2f}")

plt.figure(figsize=(8, 5))

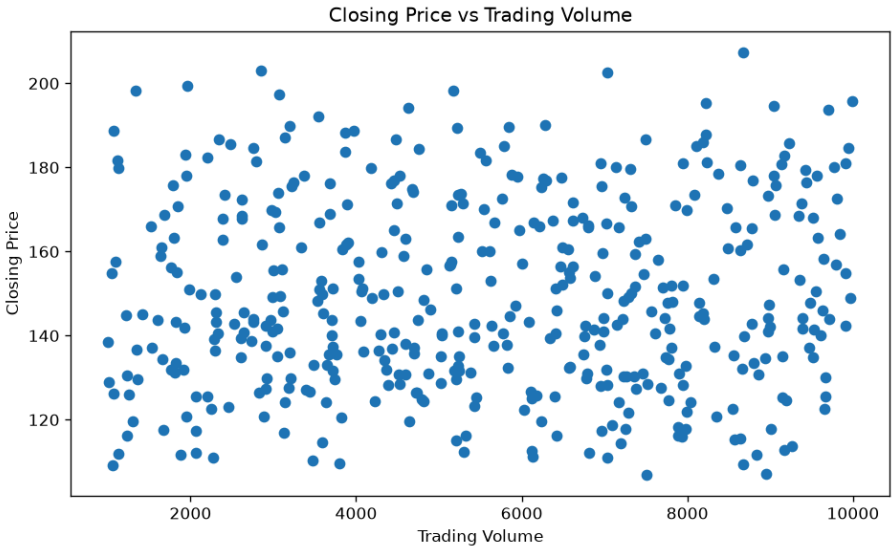
plt.scatter(df["Volume"], df["Closing_Price"])

plt.title("Closing Price vs Trading Volume")
plt.xlabel("Trading Volume")
plt.ylabel("Closing Price")
plt.tight_layout()
plt.show()
```

Output [8]:

```
Correlation between Closing Price and Volume: 0.04
```

Plot [8]:



8. Forecast future stock prices using exponential smoothing

In [9]:

```
# Use the closing price for forecasting
price_series = df["Closing_Price"]

# Create an exponential smoothing model
model = ExponentialSmoothing(
    price_series,
```

```
    trend="add",
    seasonal=None
)

model_fit = model.fit()

# Forecast the next 30 days
forecast = model_fit.forecast(30)

print("Forecasted prices:")
display(forecast.head(10))
```

Output [9]:

```
C:\Users\omkar\AppData\Local\Programs\Python\Python311\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning:
No frequency information was provided, so inferred frequency D will be used.
  self._init_dates(dates, freq)
Forecasted prices:
```

Output [9]:

```
2026-03-19    169.310719
2026-03-20    169.400714
2026-03-21    169.490710
2026-03-22    169.580706
2026-03-23    169.670701
2026-03-24    169.760697
2026-03-25    169.850692
2026-03-26    169.940688
2026-03-27    170.030684
2026-03-28    170.120679
Freq: D, dtype: float64
```

9. Plot the forecast

In [10]:

```
future_dates = pd.date_range(
    start=df.index[-1] + pd.Timedelta(days=1),
    periods=30
)

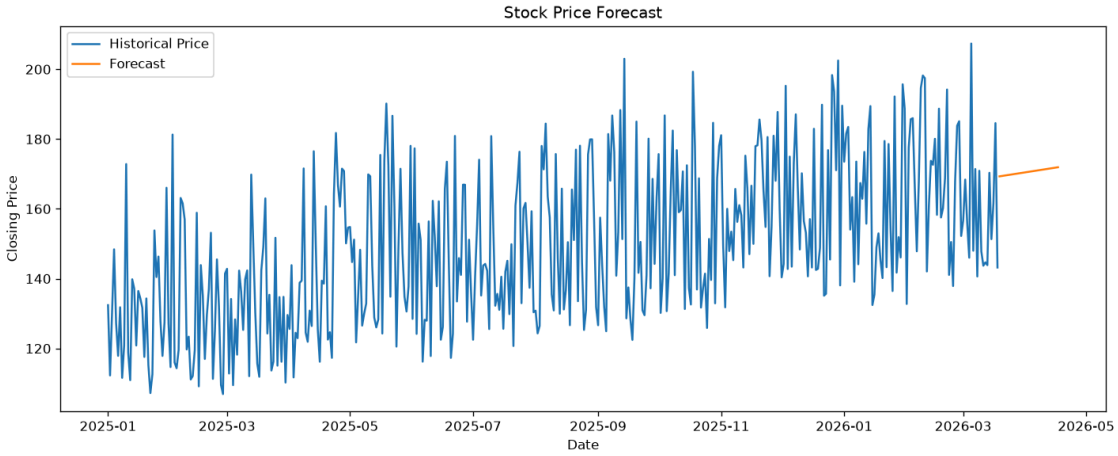
plt.figure(figsize=(12, 5))

plt.plot(
    df.index,
    df["Closing_Price"],
    label="Historical Price"
)

plt.plot(
    future_dates,
    forecast.values,
    label="Forecast"
)

plt.title("Stock Price Forecast")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.legend()
plt.tight_layout()
plt.show()
```

Plot [10]:



Conclusion

The stock-style time-series data was analyzed using:

- Date-time conversion
- Line plot for historical prices
- 7-day moving average
- Simple monthly seasonality analysis
- Correlation between price and trading volume
- Exponential smoothing for forecasting future prices

These techniques help identify trends, smooth short-term fluctuations, and make simple future-price forecasts.